

# COLX 531: Membership Attack Project: Milestone 1

Yucai Zhong

Christina McCallum

Jennifer Flake

Rachelle De Jager

March 8, 2026

## Abstract

Membership inference on clinical texts was performed by using both SmolLM2-135M and SmolLM2-360M language models, to process patient discharge notes. After both models processed the patient training data (50,000 examples), we extracted 9 features from each model, for a total of 18 features. Then, we implemented both a random forest and a logistic regression classifier. After training these two classifiers with our 18 extracted features, we used each model’s performance on the validation set to choose which model to use for the test set. Their performance was close, but the model using logistic regression had better results.

## 1 Introduction

Large language models are often fine-tuned on large datasets, which may include sensitive information such as clinical records. Membership Inference Attacks (MIA) aim to determine whether a specific data sample was used during the training or fine-tuning process of a model. If an attacker can successfully identify training data, this may raise privacy concerns, especially when the data contains confidential information.

Membership inference is an important problem because it helps evaluate the privacy risks of machine learning systems. Understanding how easily a model leaks membership information can inform better training strategies and privacy protection methods.

In this project, we study membership inference attacks using a provided dataset and pre-finetuned language models.

## 2 Task Description

The task in this project is to perform membership inference on a given dataset of clinical texts.

Specifically, we are provided with training, validation, and test splits, where each sample is labeled as either a member (1) or non-member (0) of the fine-tuning dataset. Our goal is to assign a membership score to each test sample, where higher scores indicate a higher likelihood of being a training example.

This task is important because it allows us to evaluate how vulnerable fine-tuned language models are to membership inference attacks. If models can easily reveal whether specific data was used during training, this may indicate potential privacy risks.

The objective of this project is to develop methods that can effectively distinguish between member and non-member samples and to submit predicted membership scores to the Kaggle competition for evaluation.

## 3 Data

The dataset used in this project is the `UBC-SLIME/colx_531_group_project` dataset. It contains English clinical texts, including clinical case reports and hospital discharge summaries. The dataset is designed for a Membership Inference Attack (MIA) task. In this task, the goal is to determine whether a given text sample was used to fine-tune a language model.

Each sample in the dataset is labeled with a binary variable `is_member`. If a sample was part of the model’s training data, it is labeled as `is_member = 1`; otherwise, it is labeled as `is_member = 0`. The input to the model is the clinical text, and the output is a prediction indicating whether the sample is a member of the training data.

The dataset contains three splits: a training set with 50,000 samples, a validation set with 10,000 samples, and a test set with 15,000 samples. The labels for the test set are hidden and must be predicted for submission to the Kaggle competition. The training and validation splits are balanced,

meaning that they contain approximately equal numbers of member and non-member samples.

Each data instance contains three fields: a unique identifier (`id`), the clinical text (`text`), and the membership label (`is_member`). The text samples have similar length distributions across the three splits, with an average length of around 1,950 characters.

## 4 Evaluation Metrics

### 4.0.1 Area Under the ROC Curve (AUC)

The Receiver Operating Characteristic Curve (ROC) is a graph of a classification model's True Positive Rate (TPR) on the y-axis, and its False Positive Rate (FPR) on the x-axis. TPR is the probability that the model will classify a positive label correctly as positive, and FPR is the probability that the model will classify a negative sample incorrectly as positive. The AUC is the area beneath this curve: it's a measurement from 0-1, where 1 indicates perfect performance and 0 indicates severe failure (anything from 0.5 and under is considered poor performance) (Bugbee). The AUC is calculated as an integration problem using the trapezoidal rule: divide the area under the ROC curve into multiple strips of equal width, then approximate the area of each strip, and sum these approximations (Shi-Tao, 2002).

### 4.0.2 TPR@FPR=0.1

Since AUC averages over all false positive rates, the TPR of an attack when the FPR is above 0.5 is not useful, because a high FPR means that the attacker model is just guessing. This is why the TPR at a low FPR (in this case, 0.1) is used as another evaluation metric: if an MIA can successfully identify the sensitive data of only a few users, it works well (Carlini, 2022). Once we have calculated TPR ( $TP / (TP + FN)$ ) and FPR ( $FP / (FP + TN)$ ) and graphed the ROC curve, we can find TPR@FPR=0.1 (i.e., the point on the curve where  $FPR = 0.1$ ).

## 5 Background

Summary of related research papers:

### 5.0.1 Paper 1: "Membership Inference on LLMs in the Wild" (Jiatong Yi and Yanyang Li, 2026)

This paper contributes to the research of Membership Inference Attack (MIA) through its proposal of SimMIA, a framework for evaluating MIA in the black-box context (where only model outputs are available), and WikiMIA-25, an updated dataset built for the purpose of MIA on more recent LLMs (i.e., differentiating information from 2025). The MIA framework proposed by the authors, SimMIA, involves a word-by-word sampling strategy to eliminate error propagation, and a method of semantic scoring that "captures soft membership signals beyond exact token matching" (Yi and Li, 2026) to achieve MIA results on model text output alone, as opposed to model internals like logits. The main idea is that if a text has appeared in a training set of a model, then any generated or continued text produced by that model will be more semantically consistent with the actual suffix of that text (compared to unseen texts).

The authors use the SimMIA method across two benchmarks: WikiMIA (text dumps from Wikipedia, applicable to models from 2017-2023) and MIMIR (data from The Pile dataset) across four open-source families. For WikiMIA, these are OPT-6.7B, Pythia-6.9B, LLaMA-7B, and GPT-NeoX-20B. For MIMIR, these are the Pythia-dedup family (160M, 1.4B, 2.8B, and 6.9B). They then apply their proposed method, WikiMIA-25, to newer models: Qwen3-8B-Base, Claude-4.5-Haiku, Gemini-2.5-Flash, and GPT-5-Chat. WikiMIA-25 is similar to WikiMIA, but it contains more recent Wikipedia dumps, designating events from January 2015 to December 2016 as member data, and events from March 2025 to September 2025 as non-member data. The paper's main contribution is that SimMIA outperforms standard black-box MIA baselines and logit-based approaches by an average of 15.7 AUC points, demonstrating a new method for evaluating MIA on model text outputs alone.

### 5.0.2 Paper 2: "Membership Inference Attacks against Language Models via Neighbourhood Comparison" (Mattern et al., 2023)

In the paper "Membership Inference Attacks against Language Models via Neighbourhood

Comparison," the authors propose a new method that can be applied to the task of Membership Inference Attacks. Noting that using the well-established, probabilistic methods does not take into account the inherent probabilistic variability in language, they devised a method to use synthetic, "neighbour" text that closely resembles the original training data. Using BERT for word substitution, they generated slightly modified neighbour text. Next, they computed model loss for both the original text and the neighbour text. If the original text's loss is significantly less than the neighbor text loss, the text likely appeared in the training data.

To evaluate their methods, they used three primary datasets: news article summaries from the AG News corpus, tweets from the Sentiment140 dataset, and Wikipedia article excerpts from Wikitext-103. The research employed several models, specifically using the base version of GPT-2 (117M parameters) as the target model and the pretrained 110M parameter version of BERT as the masked language model to generate synthetic neighbor samples. This paper contributes a new method of attack that is robust and outperforms other reference-free attacks. In addition, the paper shows that neighbourhood attacks' performance is competitive with traditional reference-based attacks. Lastly, strategy proposed in the paper eliminates the need for access to the original training data distribution.

### **5.0.3 Paper 3: "Membership Inference Attacks against Fine-tuned Large Language Models via Self-prompt Calibration" (Fu et al., 2024)**

The paper recognizes the fine-tuning phase of pre-trained LLMs as particularly vulnerable to privacy leakage, and aims to illustrate the prevalence of these risks through Membership Inference Attack (MIA). The authors propose a new method for MIA, based on Self-calibrated Probabilistic Variation (SPV-MIA).

The authors attempt to improve previously reported methods for MIA and outline limitations prevalent in existing reference-free and reference-based MIAs, including that both are too heavily contingent on overfitting of target models and that reference-based methods rely on access to a reference dataset that mirrors the target training set. Lastly, the method proposed in the paper provides

a theoretical explanation for why neighbor attacks work well in practice, which was not fully known.

The authors used three datasets: Wikitext-103, a text dataset with over 100 million tokens and extracted from verified articles on Wikipedia; AG News, a corpus of over 1 million news articles gathered from over 2000 news sources; and XSum, a collection of over 225,000 news articles and their corresponding summary. The authors used four open-source LLMs: GPT-2, GPT-J, Falcon-7B and LLaMA-7B.

The paper proposes SPV-MIA, a new method for MIA built on two novel mechanisms. Firstly, a Self-prompt reference model, where the target LLM is prompted to create the reference dataset. This process pulls similarly distributed data, which is a more reliable reflection of what might be available to adversarial MIAs in real-world scenarios. Secondly, a probabilistic variation metric that uses LLM memorization as the membership signal instead of overfitting. This metric detects whether a text sits at a probability peak by measuring how sharply probability drops when the text is slightly modified. The paper details experiments across the four LLMs and three datasets set out above, showing that SPV-MIA improves the AUC from approximately 0.7 to 0.9, with substantial improvements in TPR@1%FPR.

## **6 Week 1 Baseline**

Describe the baseline method implemented for the first submission.

### **6.1 Inference**

We used the SmolLM2-135M and SmolLM2-360M language models to generate logits, which are the raw, unnormalized scores the model outputs for every possible next word in the vocabulary. After both models processed the patient training data (50,000 examples), we extracted 9 features from both models, for a total of 18 features.

### **6.2 Feature Extraction**

Our feature extraction involved converting the logits from inference into probabilistic features. We applied a softmax function to the logits to turn them into probabilities (0 to 1). From those probabilities, we calculated the specific metrics that served as our features. These features included:

1. Perplexity: Low perplexity suggests that the model is more likely to have seen the item before, while high perplexity more likely means the model hasn't seen it.

2, Loss: Lower loss suggests that the model has seen this pattern before. Higher loss suggests that the model is struggling with never-seen-before patterns

3. Mean Token Probability: Mean token probability indicates the average confidence of the model across all tokens in the text. High probability indicates that it is more likely that the model has seen this pattern before. Low mean token probability indicates that the model may never have seen this pattern before.

4. Minimum Token Probability: This feature contains the lowest confidence the model had for any single token. If this statistic is high, the model is pretty confident even at the low end of its confidence. High minimum token probability can signal that it is more likely that the model has seen this data before. Low minimum token probability can signal that the model has not seen this data before.

5. Maximum Token Probability: This feature marks the highest confidence the model had for any single token. A model with a very high maximum token probability finds that some words are extremely predictable. This may be a signal that the model has seen this pattern of tokens during training.

6. Standard Deviation of Token Probability: If the data has been seen by the model during training, the standard deviation will be lower because its confidence will be higher overall. However, a higher standard deviation of token probability would indicate more variability in the model's confidence level for each token, which would suggest that it is unlikely to have seen the data during training.

7. Mean Entropy: Higher entropy means that it is less likely that the model has seen the pattern of tokens before.

8. Top k Mass (with k=10): Top k mass indicates how much probability the model assigns to its top k most confident predictions. We used k = 10. If high, top k mass signals that model has likely seen this data before. If top k mass is relatively low, the model is uncertain and likely hasn't seen this pattern before.

9. Low Confidence Token Ratio: If the ratio is closer to 0, few tokens had low confidence, which

may be a signal that the model likely saw this pattern during training. Higher low-confidence ratio (closer to 1) means that the model was frequently uncertain and likely has not seen this pattern previously.

### 6.3 Classifier Training Details

After extracting the features, we implemented two common classification algorithms: logistic regression (with feature scaling) and random forest with the 18 features that potentially signaled whether an example was a member or non-member. The models were trained on a labeled dataset of 50,000 examples.

### 6.4 Validation

The models were evaluated on a separate validation set of 10,000 examples to ensure that the attack generalized well to new data. We used each model's performance on the validation set (10,000 examples) to choose which model to use for the test set. Their performance was close, but the model using Logistic Regression had better results.

|                      |        |                   |
|----------------------|--------|-------------------|
| Logistic Regression: | AUC:   | 0.8852            |
| TPR@FPR=0.1:         | 0.6492 | Accuracy: 0.8119. |
| Random Forest:       | AUC:   | 0.8842            |
| TPR@FPR=0.1:         | 0.6554 | Accuracy: 0.8050  |

### 6.5 The Motivation Behind this Baseline

The core motivation is the "memorization signature" phenomenon. Machine learning models tend to behave differently on the data they have seen during training compared to the data they are seeing for the first time. By analyzing the confidence and uncertainty of the model, we hoped to detect these subtle differences.

In addition, we used two language models of different sizes (135M and 360M) for inference with the theory that they would capture varying degrees of memorized patterns. Larger models often memorize data more effectively than smaller ones.

Finally, we took advantage of Google Colab's compute capabilities to use the entire 50,000 item training set both for inference and training. We implemented both random forest and logistic regression models, so that we could compare the two and use the higher performing model.

## 6.6 Hardware Used

To handle the heavy computational load of processing 50,000 text samples through two transformer models, we utilized high-performance GPU hardware on Google Colab:

- **GPU:** NVIDIA A100-SXM4-40GB.
- **GPU Memory:** 39.5 GB of VRAM, which allowed for efficient batch processing of the 360M parameter model.
- **Framework:** The pipeline was built using `torch` (PyTorch) and the Hugging Face `transformers` library to leverage CUDA acceleration.

## 6.7 The Pipeline

The pipeline followed a four-stage process:

1. **Inference:** Discharge notes were passed through both SmolLM2 language models. We captured the logits for every token.
2. **Statistical Calculation:** Using the logits, we calculated probabilistic metrics.
3. **Classification:** These metrics were fed into the logistic regression and random forest classifiers.
4. **Performance:** The logistic regression classifier final baseline achieved an AUC of 0.8852 and an Accuracy of 81.19%, indicating a high degree of success in identifying which notes were part of the training set.

## 6.8 Kaggle results / observations

Our Kaggle score for the competition this week is 0.89291. While we are very pleased with the model's preliminary results, we plan to continue to refine it for improved results.

## 6.9 Conclusion

Our model had strong performance for a baseline. Our future work will include implementing methods and features that we have learned from the literature we have read this week.

## 7 Bibliography

Carlini, Nicholas, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. 2022. Membership inference attacks from first principles. *darkblue-https://doi.org/10.48550/arXiv.2112.03570*.

*Cryptography and Security*, darkbluearXiv:2112.03570v2. Version 2.

Bugbee, Erin, Anand Kamat, and Jared Wilber. n.d. ROC & AUC. MLU-Explain, Amazon Machine Learning University. Retrieved March 7, 2026, from <https://mlu-explain.github.io/roc-auc/>.

Wenjie Fu, Huandong Wang, Chen Gao, Guanghua Liu, Yong Li, and Tao Jiang. 2024. Membership Inference Attacks against Fine-tuned Large Language Models via Self-prompt Calibration. [https://proceedings.neurips.cc/paper\\_files/paper/2024/file/f360Paper-Conf.pdf](https://proceedings.neurips.cc/paper_files/paper/2024/file/f360Paper-Conf.pdf). *Advances in Neural Information Processing Systems*, 37:134981–135010.

Mattern, Justus, Fatemehsadat Mireshghallah, Zhijing Jin, Bernhard Schölkopf, Mrinmaya Sachan, Taylor Berg-Kirkpatrick. 2023. Membership Inference Attacks against Language Models via Neighbourhood Comparison. <https://aclanthology.org/2023.findings-acl.719.pdf>. *Findings of the Association for Computational Linguistics: ACL 2023*, 11330–11343.

Shi-Tao Yeh. 2002. Using trapezoidal rule for the area under a curve calculation. In *Proceedings of the 27th Annual SAS Users Group International Conference (SUGI 27)*, paper 229-27. SAS Institute Inc.

Yi, Jiatong and Yanyang Li. 2026. Membership inference on LLMs in the wild. *darkblue-https://doi.org/10.48550/arXiv.2601.11314*. *Computation and Language*, darkbluearXiv:2601.11314v1. Version 1.

## 8 Contributions

### 8.1 Milestone 1

Christina: Read "Membership Inference on LLMs in the Wild" and wrote the summary for it. Researched the evaluation metrics and wrote the summary for that section. 22%

Jennifer: Implemented and documented baseline model. Submitted test results to Kaggle. Read "Membership Inference Attacks against

Language Models via Neighbourhood Comparison" and wrote the summary for it. 34%

Rachelle: Setup group git repo and drafted teamwork contract. Read "Membership Inference Attacks against Fine-tuned Large Language Models via Self-prompt Calibration" and wrote the summary. 22%

Yucai: Set up the overleaf template, wrote the task description and data description, and implemented data inspection. 22%